

Spring 2026

Data Structures Section D11 — Lab 09

Friday May 15, 2026

Topic

Recursive functions

Objective

- Write Recursive functions

Outcomes

1. Implement and use Recursive Functions

Content

The following sections will be covered during this lab session.

1. **Code and test Recursive Functions**

Tasks

Students are required to complete the following tasks in lab timings.

For the tasks 1-5, create a private repository in your account on Github: **S26_DSA_D20_Lab_9**. Upload codes with the names **Task_1**, **Task_2** and so on it.

Mandatory for Each Task

The following Steps must be **included** as **commented text** in **each .cpp file of each task**, before function main. **Missing these steps mean that you have NOT done the work yourself and it is from ChatGPT. (Please use Plain English or Urdu and No C++ Coding terms)**

Please note that these steps are more important than writing code, even at the expense of code incompleteness.

1. **Clearly and in details**, explain what you understand about the task at hand after carefully reading the given description and code.
2. Describe how will you do the task and the **solution** e.g. how to code a given functions or solution to the problem. You should use detailed steps and use the notations given in the slides for the LinkedList e.g. (A → B → C →) to show the progression of the solution.
3. Describe what **functions** you will **reuse** and **why**. Furthermore, explain the solution in terms of functions used. (**make sure your code is as per your solutions steps**). **Show** LinkedList e.g. (A → B → C →) to show the progression of the LinkedList after each function call.
4. Describe **your learning** from the task
5. Describe the **skills** you have developed and what type of problems you can code based on your experience of doing the task

Header files

The header files have been provided as a zip file and these files must be used in your lab tasks

Previous Lab Remaining Tasks

First Complete the **remaining** tasks of the previous Lab before starting with the following tasks

Task 1

Write recursive code for the following functions, You may use more helper functions but all the functions must be recursive and use a suitable main to test all these functions:

Can't change the prototype of the functions, unless required and explained in comments

1. void countWords(const char * sentence); // Count and display number of words in the sentence
2. void countCharaters(const char * sentence); //Count and display number of characters
3. void reverse(char * & sentence); // reverses the sentence character- wise
4. void reverseWords(char * & sentence); // reverses the sentence words-wise
5. void convert(char * sentence); // converts lower to uppercase and uppercase to lowercase

Task 2

In this task, your will write recursive functions on ArrayStack. You may use more helper functions but all the functions must be recursive and use a suitable main to test all these functions:

Can't change the prototype of the functions, unless required and explained in comments

1. int evaluatePrefix(char * expression, const ArrayStack<int> s);// evaluates the prefix math expression, assume single digits no brackets and no precedence rules
2. int evaluatePostfix(char * expression, ArrayStack<int> s);// evaluates the postfix math expression, assume single digits no brackets and no precedence rules
3. void copyTOH(ArrayStack<int> src, ArrayStack<int> & copy); // follow the Tower of Hanoi rules of copy: **(Stack can be of any size)**
 - a. The values are in order: smaller values are on top of larger values
 - b. When moving to any temporary Stack, larger values can't be on top of a smaller value

Task 4

In this task, your will write recursive functions on ArrayQueue. You may use more helper functions but all the functions must be recursive and use a suitable main to test all these functions: **(some of these functions are part your assignment 4)**

Can't change the prototype of the functions, unless required and explained in comments

1. void insert(ArrayQueue<int> & q, int value, int position);// insert value at position
2. void deleteValue(ArrayQueue<int> & q, int position);// delete value at position
3. bool isPalindrome(const ArrayQueue <int> & q);// assume single digits and no other queue or any other ADT is allowed

Task 5

In this task, your will write recursive functions on DoublyLinkedList. You may use more helper functions but all the functions must be recursive and use a suitable main to test all these functions: **(some of these functions are part your assignment 4)**

Can't change the prototype of the functions, unless required and explained in comments

1. void insert(DoublyLinkedList <int> & list, int value, int position);// insert value at position
2. void deleteValue(DoublyLinkedList <int> & list, int position);// delete value at position
3. bool isPalindrome(const DoublyLinkedList <int> & list);// assume single digits and no other ADT is allowed

Task 6

In this task, you will write recursive functions on BinarySearchTree. You may use more helper functions but all the functions must be recursive and use a suitable main to test all these functions: **(some of these functions are part your assignment 4). Please use the following Class Node.**

```
template <class T>
class Node
{
public:
    T data;
    Node * left;
    Node * right;
};
```

Can't change the prototype of the functions, unless required and explained in comments

1. int height(Node * treeRoot); returns height of the tree
2. int countNodes(Node * treeRoot); returns number of Nodes in the tree
3. int countLeaves(Node * treeRoot); returns number of nodes in the tree
4. Node * copyTree(Node * treeRoot); returns a deep copy of the tree
5. void inorderDisplay(Node * root);// Display all Tree Nodes using inorder traversal
6. bool preorderSearch(Node * root, T value);// search value using preorder traversal
7. bool preorderSearch(Node * root, T value);// search value in the **BinarySearchTree**